BHASHINI NAVIGATOR

1. Problem Statement and Knowledge

You need to read traffic signs and bus and metro boards, along with railway signs, shop boards, pedestrian signals and exit signs to find your way around a city. But it is not always easy to do this when the font is too small, the language is not known or the sign is not clearly visible. Blind and low-vision users can't read signs visually either. Information is increasingly provided in audio form, which deaf or hard-of-hearing users cannot access, and people with speech or hand-motor impairments and senior users have trouble using mainstream digital assistance.

We are they thinking that an equitable solution would be to facilitate asking for help and understanding the response. Sign information should be translated into simple, actionable instructions in a language and medium that is suitable for each individual. This is our Project on " Navigating City Signage " Use Case. Our major mandatory required condition is visual impairment and some hearing, speech, hand-motor, and age related conditions would be covered as well.

2. Idea / Abstract

The Bhashini Navigator is a multilingual web assistant that allows users to read signs and get basic directions in the place they are. Users can capture an image of a sign or upload a photo. The text will be detected, translated to the target language, and given meaning in plain language. The explanation can be read on screen or heard in speech. You can ask questions by speaking or typing, or you can select from a large list of predefined common phrases.

We aim to support English, Telugu and Hindi, subject to the languages supported by the configured services. We will integrate Bhashini's speech recognition, translation and text-to-speech services to enable voice-based interaction. The interface will automatically adjust to users' chosen accessibility needs, including screen-reader support, captions, transcripts, larger controls and customizable speech. In case of ambiguity in a sign, the system will ask the user to verify the text or to take another picture, rather than guessing.

3. High-Level Approach and Architecture

The frontend will be a responsive web interface where users can take or upload signs, select a language and customize accessibility options. A Node.js/Express backend will authenticate requests and communicate securely with Bhashini and the configured OCR or AI services. API credentials will be stored on the server, not in frontend code. Mapping or routing services may be added for nearby destinations if configured and available. The system will explicitly identify demo content and will not generate routes or distances.

Planned User-Facing Features

- Visual impairment: screen-reader-friendly results, spoken explanations, repeat/slow speech and prompts to retake a photo.
- Hearing impairment: captions, readable transcripts and visual alerts. Speech impairment: type questions editable text and a large phrase board.
- Hand-motor impairment: large buttons, keyboard support, fewer steps and clear confirm/cancel actions.
- Ageing: larger text, high contrast, simple language and step-by-step instructions.
- In case of failure: offer retry, text correction, a new photo or a clear explanation of unavailable services.

4. Planned Testing Methodology

We will perform functional, integration, accessibility, language and reliability testing on the prototype. We will test against the actual services where credentials and network access are available. In cases where they are not available, we will test for error handling and other features using clearly labelled test data. Demo responses will not be presented as real Bhashini results.

A. Functional and End-to-End Tests

- Capture or upload a sign and verify the preview, text recognition, validation, retake option and error messages.
- Test with traffic, bus, metro or railway, shop, exit and emergency signs, including blurry or partially visible images.
- Ensure the entire process from recognized text to translation, simple explanation and text/audio output works correctly.
- Test voice, typing and phrase-board questions, language changes, scan history and saved preferences.

Validate that navigation results appear only when a real routing service is available, otherwise, the limitation is displayed.

B. Bhashini and API Integration Tests

Test speech recognition, translation and text-to-speech for supported English, Telugu and Hindi language combinations with valid credentials.

Test authentication, request and response formats, timeouts, rate limits and network failures.

Monitor behavior when credentials are missing or invalid, including a relevant status message and clear indication of demo mode. Confirm API keys and other secrets are not exposed in frontend code, logs or error messages.

C. Accessibility Tests

Visual impairment: check keyboard and screen-reader navigation, read results aloud and provide easily understandable photo-retaking instructions.

Hearing impairment: ensure there are captions and transcripts for produced audio responses.

Speech impairment: perform critical tasks using typing and phrase-board options without having to speak.

For the motor-handicapped: test large buttons, keyboard access, and scenarios that don't require fine motor control.

Ageing: test larger text, readable contrast, plain-language explanations and step-wise guidance.

Test each mode on mobile and desktop, separately and simultaneously, in addition to visibility of focus and reduced-motion preferences.

D. Reliability and Acceptance Criteria

Run the frontend build, start the backend and run all type checks, linting, tests and health checks that are available to you.

Invalid uploads, languages not supported, blurry images, speech recognition errors, audio malfunctioning, service goes down.

Make sure that crucial buttons function, user input is preserved in the event of recoverable errors, and information of uncertain veracity isn't presented as fact.

Given the credentials, show a live Bhashini speech-to-speech session and compare the original sign text with what is provided in an accessible format.

Document the scenarios tested, issues discovered, corrections made, and limitations that still exist. Don't mention the results of a real-user test unless it really happened. Output Expectation

We want to provide a functional, interactive prototype with which people can experience reading city signs with multi-language text recognition, translation, basic explanations, and accessible speech or text. The prototype will feature specific support for visual, hearing, speech, hand motor and ageing related requirements. It also manages uncertain results with care, informing the end user regarding any limitations of availability of external services.

Expected Outcome

We aim to deliver a working, responsive prototype that helps people understand city signs through multilingual text recognition, translation, simple explanations and accessible speech or text.